Algorithm Interview Reference — Theory Guide

Core Algorithms, Data Structures, and Implementation Techniques Generated on: January 06, 2026 Content: 16 comprehensive algorithm theory guides with implementations, optimizations, and complexity analysis

---

> → **Key Idea**: Efficiently track connected components and detect cycles in undirected graphs using path compression and union by rank for nearly constant time operations.

> • **When to use**: Detect cycles in undirected graphs, find connected components, dynamic connectivity queries, Kruskal's MST algorithm

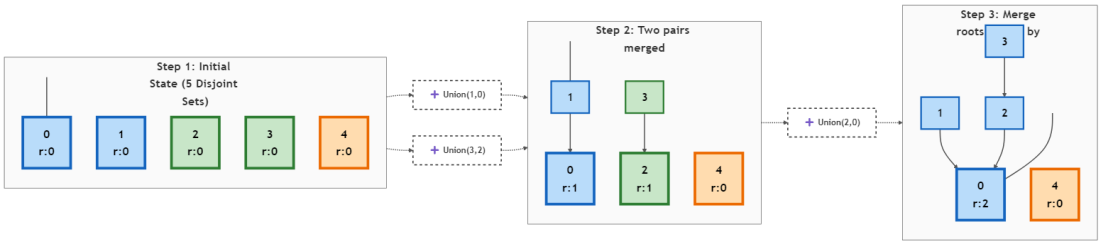
> ⊕ **Complexity**: Time O(α(n)) per operation with optimizations, Space O(n)

> ⚠ **Common Pitfall**: Forgetting path compression leads to O(n) per operation instead of O(α(n))

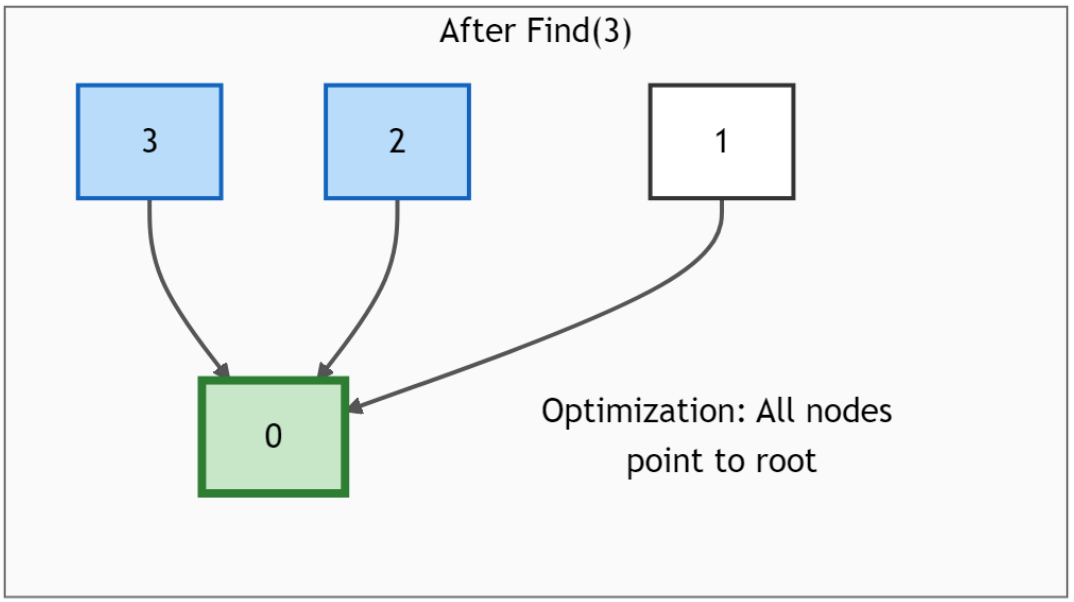
Overview

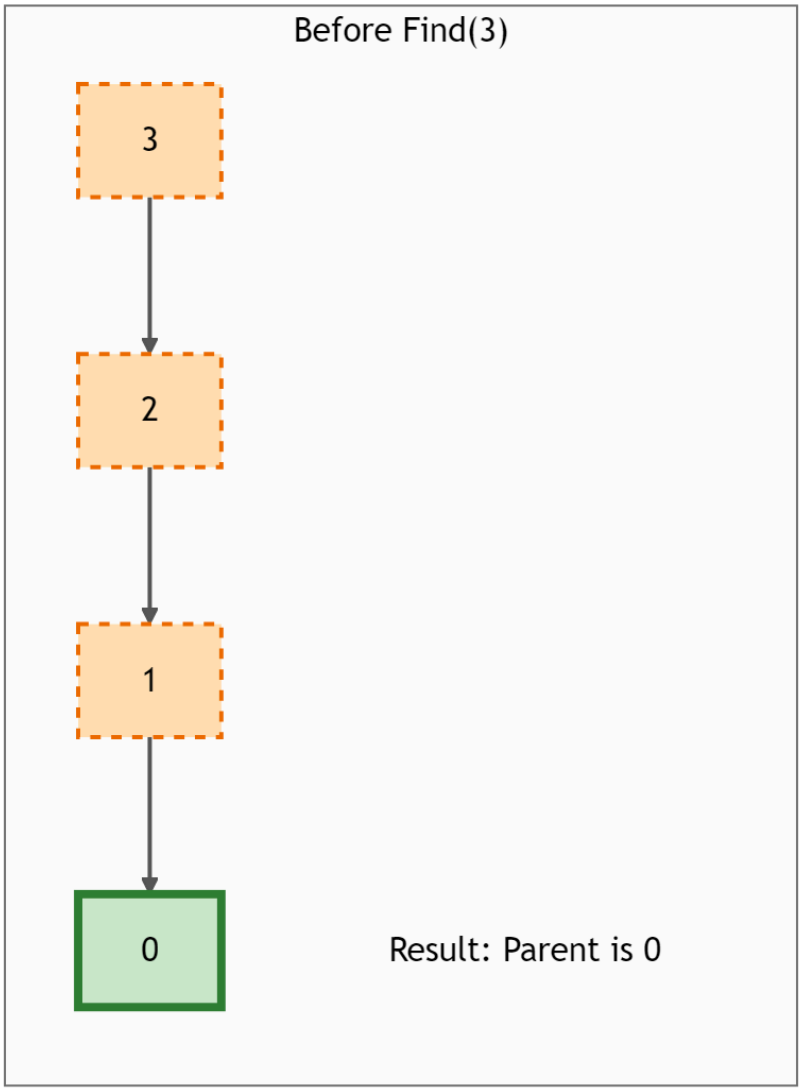
Visual Example

Union-Find Tree Evolution



Path Compression Optimization





Core Implementation

```
public class UnionFind {
    private int[] parent, rank;
    private int count; // Number of connected components

    public UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        count = n;

        // Initialize: each node is its own parent
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            rank[i] = 0;
        }
    }

    /**
     * Find the root of the set containing element i
     * Uses path compression for optimization
     */
    public int find(int i) {
        if (parent[i] != i) {
            parent[i] = find(parent[i]); // Path compression
        }
        return parent[i];
    }

    /**
     * Union two sets containing elements i and j
     * Uses union by rank for balanced trees
     */
    public void union(int i, int j) {
        int rootI = find(i);
        int rootJ = find(j);

        if (rootI != rootJ) {
            // Union by rank: attach smaller tree under root of deep
            if (rank[rootI] > rank[rootJ]) {
                parent[rootJ] = rootI;
            } else if (rank[rootI] < rank[rootJ]) {
                parent[rootI] = rootJ;
            } else {
                parent[rootJ] = rootI;
                rank[rootI]++; // Increase rank when trees have same
            }
```

```
                count--; // Decrease number of connected components
            }
        }

        /**
         * Check if two elements are in the same set
         */
        public boolean connected(int i, int j) {
            return find(i) == find(j);
        }

        /**
         * Get number of disjoint sets
         */
        public int getCount() {
            return count;
        }
    }
```

Key Optimizations

1. Path Compression

```
// Without path compression (slow)
public int findSlow(int i) {
    while (parent[i] != i) {
        i = parent[i];
    }
    return i;
}

// With path compression (fast)
public int find(int i) {
    if (parent[i] != i) {
        parent[i] = find(parent[i]); // Compress path
    }
    return parent[i];
}
```

2. Union by Rank

```
// Without union by rank (can create tall trees)
public void unionSlow(int i, int j) {
    int rootI = find(i);
    int rootJ = find(j);
    if (rootI != rootJ) {
        parent[rootJ] = rootI; // Always attach j to i
    }
}

// With union by rank (balanced trees)
public void union(int i, int j) {
    int rootI = find(i);
    int rootJ = find(j);
    if (rootI != rootJ) {
        if (rank[rootI] > rank[rootJ]) {
            parent[rootJ] = rootI;
        } else if (rank[rootI] < rank[rootJ]) {
            parent[rootI] = rootJ;
        } else {
            parent[rootJ] = rootI;
            rank[rootI]++;
        }
    }
}
```

> → **Key Idea**: Search sorted array by repeatedly dividing search space in half

> • **When to use**: Sorted arrays, search spaces, optimization problems, finding boundaries

> ⊕ **Complexity**: Time O(log n), Space O(1)

> ⚠ **Common Pitfall**: Use `mid = left + (right - left) / 2` to avoid integer overflow

Overview

Visual Example

Binary Search Process

```
Array: [1, 3, 5, 7, 9, 11, 13, 15, 17, 19]  Target: 7
Step 1: [1, 3, 5, 7, 9, |11|, 13, 15, 17, 19]  mid=11 > 7, go left
Step 2: [1, 3, |5|, 7, 9] 11, 13, 15, 17, 19   mid=5 < 7, go right
Step 3:  1, 3, 5, |7|, 9  11, 13, 15, 17, 19   mid=7 = 7, FOUND!

>> Each step eliminates half the search space: 10 → 5 → 2 → 1
```

Search Space Visualization

```
Lower Bound vs Upper Bound:

Array: [2, 2, 2, 5, 5, 8, 8, 8, 8]  Target: 5

Lower Bound (first ≥ target):    Upper Bound (first > target):
[2, 2, 2, |5, 5|, 8, 8, 8, 8]   [2, 2, 2, 5, 5, |8|, 8, 8, 8]
         ↑                                  ↑
      index 3                            index 5

Range of target 5: [3, 4]
```

Binary search is a search algorithm that finds the position of a target value within a sorted array. It compares the target value to the middle element and eliminates half of the array from consideration.

The key insight is that by comparing the target with the middle element, we can determine which half of the array to search next, effectively halving the search space in each iteration.

Core Implementation

Standard Binary Search

```java
public int binarySearch(int[] nums, int target) {
    int left = 0, right = nums.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2; // Avoid overflow

        if (nums[mid] == target) {
            return mid; // Target found
        } else if (nums[mid] < target) {
```

```
            left = mid + 1; // Target is in right half
        } else {
            right = mid - 1; // Target is in left half
        }
    }

    return -1; // Target not found
}
```

> → **Key Idea**: Use two pointers moving towards each other or in same direction to solve array problems efficiently

> • **When to use**: Sorted arrays, finding pairs, removing duplicates, palindromes, sliding patterns

> ⊕ **Complexity**: Time O(n), Space O(1)

> ⚠ **Common Pitfall**: Not handling edge cases like empty arrays or single elements

Overview

Visual Example

Two Pointers Technique

```
Target Sum Problem: Find pair that sums to 9
Array: [1, 3, 4, 6, 8, 9, 11]

Step 1: left=0, right=6
[1, 3, 4, 6, 8, 9, 11]    1 + 11 = 12 > 9, move right pointer left
 ↑                 ↑

Step 2: left=0, right=5
[1, 3, 4, 6, 8, 9, 11]    1 + 9 = 10 > 9, move right pointer left
 ↑              ↑

Step 3: left=0, right=4
[1, 3, 4, 6, 8, 9, 11]    1 + 8 = 9 ✓ FOUND!
 ↑           ↑
```

```
⚡ O(n) solution vs O(n²) brute force
```

Fast & Slow Pointers (Cycle Detection)

```
Linked List with Cycle:
1 → 2 → 3 → 4 → 5
    ↑           ↓
    8 ← 7 ← 6 ←

Step 1: slow=1, fast=2     Step 2: slow=2, fast=4     Step 3: slow=3, fa
Step 4: slow=4, fast=8     Step 5: slow=5, fast=3     Step 6: slow=6, fa
Step 7: slow=7, fast=5     Step 8: slow=8, fast=2     Step 9: slow=2, fa

>> When slow == fast, cycle detected!
```

The two-pointer technique uses two pointers that traverse the data structure in a coordinated way. The pointers can move toward each other (opposite direction) or in the same direction at different speeds.

This technique is particularly powerful because it often reduces time complexity from $O(n^2)$ to $O(n)$ while maintaining $O(1)$ space complexity.

Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics
- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid
- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
// Template code for common variations
```

Time & Space Complexity

Pattern	Time Complexity	Space Complexity
Two Sum (sorted)	O(n)	O(1)
Three Sum	$O(n^2)$	O(1)
Cycle Detection	O(n)	O(1)
Array Partitioning	O(n)	O(1)
Container Water	O(n)	O(1)

The two-pointer technique typically achieves linear time complexity with constant space, making it highly efficient for interview problems.

← Previous: Binary Search | Back to Main Guide | Next: Sliding Window →

> → **Key Idea**: Use a window that slides over array to track subset properties efficiently

> • **When to use**: Subarray problems, substring problems, fixed/variable window size optimization

> ⊕ **Complexity**: Time O(n), Space O(k) where k is window constraint

> ⚠ **Common Pitfall**: Not properly updating window state when sliding or expanding/contracting

Overview

Visual Example

Fixed Size Sliding Window

```
Find maximum sum subarray of size 3:
Array: [2, 1, 5, 1, 3, 2]

Window 1: [2, 1, 5] | 1, 3, 2    Sum = 8
Window 2:  2, [1, 5, 1] | 3, 2    Sum = 7
Window 3:  2, 1, [5, 1, 3] | 2    Sum = 9  ← Maximum
Window 4:  2, 1, 5, [1, 3, 2]     Sum = 6

🪟 Slide window one position at each step
```

Variable Size Sliding Window

```
Longest substring with at most 2 distinct characters:
String: "eceba"

Step 1: [e]           distinct=1 ≤ 2 ✓, expand
Step 2: [ec]          distinct=2 ≤ 2 ✓, expand
Step 3: [ece]         distinct=2 ≤ 2 ✓, expand
Step 4: [eceb]        distinct=3 > 2 ✗, shrink
Step 5:  e[ceb]       distinct=3 > 2 ✗, shrink
Step 6:  ec[eb]       distinct=2 ≤ 2 ✓, expand
Step 7:  ec[eba]      distinct=3 > 2 ✗, done

>> Best window: "ece" (length 3)
```

Timeline Visualization

```
Time:    0----1----2----3----4----5----6
Window:  [----][----][----][----]
Action:   add   add   add   rem   add   add
State:    1     2     3     2     3     4

>> Window expands/contracts based on constraints
```

The sliding window technique involves maintaining a "window" of elements and sliding it across the data structure to find subarrays/substrings that satisfy certain conditions.

This technique is particularly powerful for optimization problems on contiguous sequences, often reducing time complexity from $O(n^2)$ to $O(n)$.

Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics
- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid
- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
// Template code for common variations
```

Time & Space Complexity

Pattern	Time Complexity	Space Complexity
Fixed Window	O(n)	O(1)
Variable Window	O(n)	O(k) where k is unique elements
Character Frequency	O(n)	O(1) or O(alphabet size)
Sliding Window Maximum	O(n)	O(k)

The sliding window technique typically achieves linear time complexity, making it highly efficient for subarray/substring optimization problems.

← Previous: Two Pointers | Back to Main Guide | Next: Monotonic Stack →

> → **Key Idea**: Use stack that maintains elements in monotonic order to find next/previous greater/smaller elements

> • **When to use**: Next greater element, histogram problems, temperature problems, expression evaluation

> ⊕ **Complexity**: Time O(n), Space O(n)

> ⚠ **Common Pitfall**: Not handling empty stack or maintaining wrong monotonic order

Overview

Visual Example

Monotonic Stack Operations

```
Next Greater Element: [2, 1, 2, 4, 3, 1]

Step 1: Process 2        Step 2: Process 1        Step 3: Process 2
Stack: [2]              Stack: [2, 1]           Stack: [2]
                                                NGE[1] = 2

Step 4: Process 4        Step 5: Process 3        Step 6: Process 1
Stack: []               Stack: [4, 3]           Stack: [4, 3, 1]
NGE[2] = 4
NGE[2] = 4

Final Result: [4, 2, 4, -1, -1, -1]

>> Stack maintains decreasing order, pops when greater element found
```

Stack State Evolution

```
Input:  [2, 1, 2, 4, 3, 1]
        ↓

Step by step:
|     |    |     |    |  2  |    |  4  |    |  4  |    |  4  |
|     | -> |  2  | -> |  1  | -> |     | -> |  3  | -> |  3  |
|     |    |  1  |    |     |    |     |    |     |    |  1  |
```

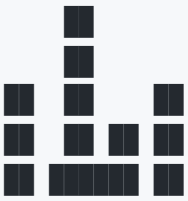
```
+-----+    +-----+    +-----+    +-----+    +-----+    +-----+
  Empty      Add 2     Add 1     Pop all    Add 3,4    Add 1

>> Monotonic decreasing stack for Next Greater Element
```

Histogram Area Calculation

```
Heights: [2, 1, 5, 6, 2, 3]

Visual representation:
```



```
Largest rectangle: Height 5, Width 2 → Area = 10

Stack helps find left and right boundaries efficiently
```

A monotonic stack maintains elements in monotonic order (either strictly increasing or decreasing). It's used to solve problems that require finding the next or previous greater/smaller element efficiently.

The key insight is that when we encounter an element that violates the monotonic property, we can process all elements in the stack that are smaller/larger before adding the current element.

Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics
- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid
- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
// Template code for common variations
```

Time & Space Complexity

Problem Type	Time Complexity	Space Complexity
Next/Previous Element	O(n)	O(n)
Daily Temperatures	O(n)	O(n)
Histogram	O(n)	O(n)
Expression Evaluation	O(n)	O(n)

Each element is pushed and popped at most once, giving O(n) amortized time complexity.

← Previous: Sliding Window | Back to Main Guide | Next: Intervals →

> → **Key Idea**: Sort intervals and merge overlapping ones to solve scheduling and range problems

> • **When to use**: Meeting rooms, scheduling conflicts, range merging, calendar problems

> ⊕ **Complexity**: Time O(n log n) for sorting, Space O(n)

> ⚠ **Common Pitfall**: Not sorting intervals first or incorrect overlap detection logic

Overview

Visual Example

Interval Merging Process

```
Input intervals: [[1,3], [2,6], [8,10], [15,18]]

Step 1: Sort by start time (already sorted)
[1,3] [2,6] [8,10] [15,18]

Step 2: Merge overlapping intervals
[1,3] overlaps with [2,6] → merge to [1,6]
[1,6] no overlap with [8,10] → keep separate
[8,10] no overlap with [15,18] → keep separate

Result: [[1,6], [8,10], [15,18]]

Timeline visualization:
1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
[──────────────────]               [──────────]               [──────
    Merged                            Kept                      Kept
```

Meeting Room Scheduling

```
Meeting times: [(0,30), (5,10), (15,20)]

Timeline:
0    5   10   15  20    30
├────┼────┼────┼──┼─────┤
│████████████████████████│ Meeting 1: (0,30)
│     ████████           │ Meeting 2: (5,10)  ← Conflicts!
│          ████████      │ Meeting 3: (15,20) ← Conflicts!

🔥 Conflicts detected → Need multiple rooms
```

Interval Intersection

```
A: [1,3], [5,7], [9,11]
B: [2,4], [6,8], [10,12]

Visual intersection:
1   2   3   4   5   6   7   8   9  10  11  12
[─────────]         [─────────]     [─────────]  A
    [─────────]         [─────────]     [─────────]  B
    [──]                [─]             [─]          Intersections
```

```
Result: [[2,3], [6,7], [10,11]]
```

Interval problems involve working with ranges of values. The key insight is usually to sort intervals by their start time and then process them in order.

Common patterns include merging overlapping intervals, finding non-overlapping intervals, and scheduling problems.

Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics
- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid
- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
// Template code for common variations
```

Time & Space Complexity

Problem Type	Time Complexity	Space Complexity
Merge Intervals	O(n log n)	O(n)
Insert Interval	O(n)	O(n)
Meeting Rooms	O(n log n)	O(n)
Calendar Problems	O(n) per operation	O(n)

Most interval problems require O(n log n) time due to sorting, with O(n) space for the result.

← Previous: Monotonic Stack | Back to Main Guide | Next: Top-K & Heaps →

> → **Key Idea**: Use heaps to efficiently maintain top-k elements or find extremes in streaming data

> • **When to use**: Finding top-k elements, streaming data, priority scheduling, median finding

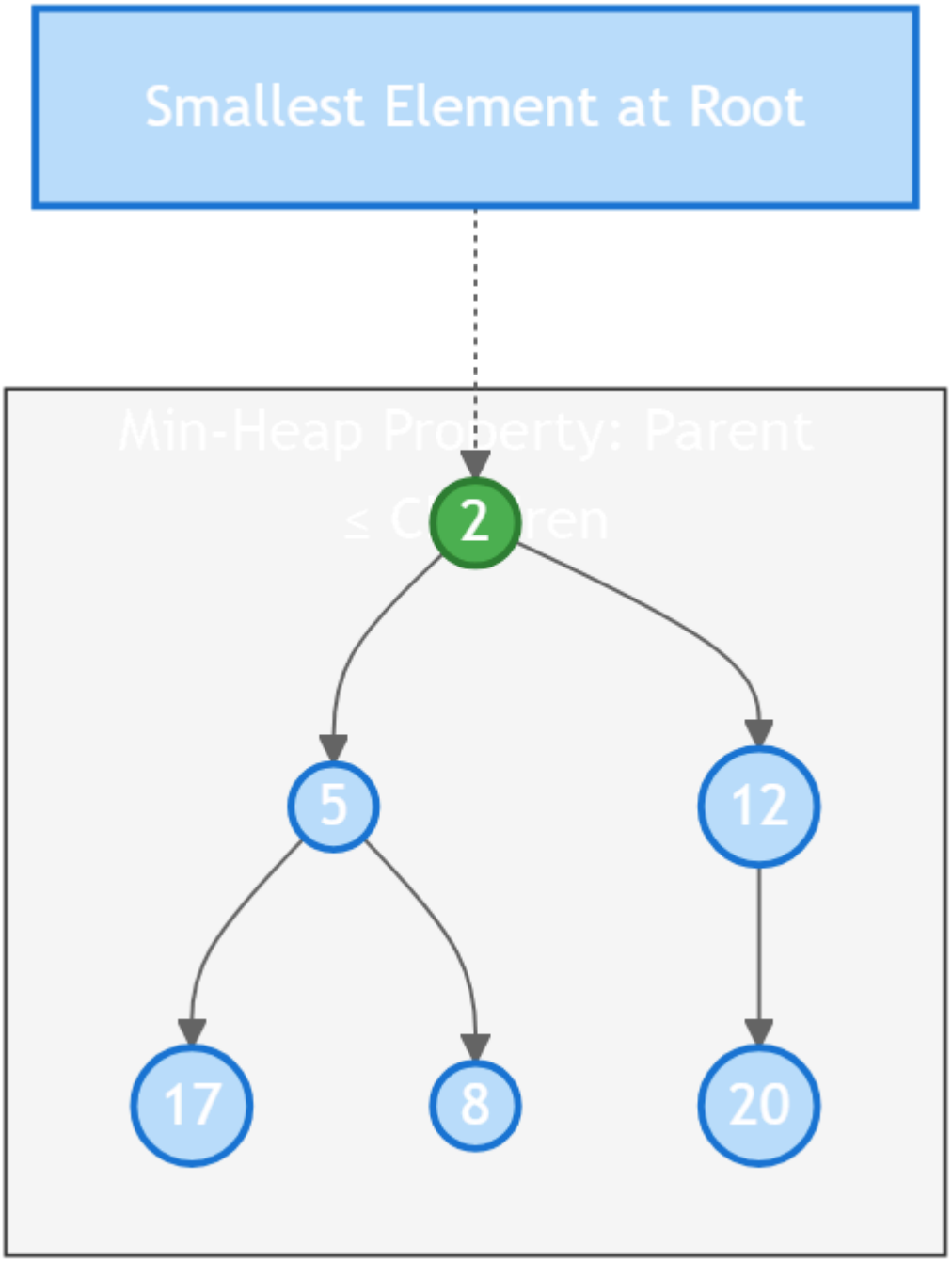
> ⨁ **Complexity**: Time O(n log k) for top-k, Space O(k)

> ⚠ **Common Pitfall**: Using wrong heap type (min vs max) or not maintaining heap size

Overview

Visual Example

Min-Heap Operations Visualization



Core Implementation

Basic Heap Implementation

```
import java.util.*;

public class MinHeap {
    private List<Integer> heap;
```

```
    public MinHeap() {
        heap = new ArrayList<>();
    }

    public void insert(int val) {
        heap.add(val);
        heapifyUp(heap.size() - 1);
    }

    public int extractMin() {
        if (heap.isEmpty()) {
            throw new IllegalStateException("Heap is empty");
        }

        int min = heap.get(0);
        int lastVal = heap.remove(heap.size() - 1);

        if (!heap.isEmpty()) {
            heap.set(0, lastVal);
            heapifyDown(0);
        }

        return min;
    }

    public int peek() {
        if (heap.isEmpty()) {
            throw new IllegalStateException("Heap is empty");
        }
        return heap.get(0);
    }

    private void heapifyUp(int index) {
        while (index > 0) {
            int parentIndex = (index - 1) / 2;

            if (heap.get(index) >= heap.get(parentIndex)) {
                break;
            }

            swap(index, parentIndex);
            index = parentIndex;
        }
    }

    private void heapifyDown(int index) {
        while (true) {
            int smallest = index;
```

```
            int leftChild = 2 * index + 1;
            int rightChild = 2 * index + 2;

            if (leftChild < heap.size() &&
                heap.get(leftChild) < heap.get(smallest)) {
                smallest = leftChild;
            }

            if (rightChild < heap.size() &&
                heap.get(rightChild) < heap.get(smallest)) {
                smallest = rightChild;
            }

            if (smallest == index) {
                break;
            }

            swap(index, smallest);
            index = smallest;
        }
    }

    private void swap(int i, int j) {
        int temp = heap.get(i);
        heap.set(i, heap.get(j));
        heap.set(j, temp);
    }

    public int size() {
        return heap.size();
    }

    public boolean isEmpty() {
        return heap.isEmpty();
    }
}
```

Priority Queue Using Java Collections

```
import java.util.PriorityQueue;
import java.util.Collections;

public class HeapExamples {

    // Min-heap (default in Java)
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
```

```
    // Max-heap (reverse order)
    PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections

    // Custom comparator
    PriorityQueue<int[]> customHeap = new PriorityQueue<>((a, b) -> 

    // String priority queue (lexicographic order)
    PriorityQueue<String> stringHeap = new PriorityQueue<>();
}
```

Key Optimizations

1. Top-K Elements Optimization

```
public List<Integer> topKElements(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();

    for (int num : nums) {
        minHeap.offer(num);

        if (minHeap.size() > k) {
            minHeap.poll(); // Remove smallest
        }
    }

    return new ArrayList<>(minHeap);
}

// Alternative: Use selection algorithm for O(n) average case
public int quickSelect(int[] nums, int k) {
    return quickSelectHelper(nums, 0, nums.length - 1, nums.length -
}

private int quickSelectHelper(int[] nums, int left, int right, int t
    if (left == right) return nums[left];

    int pivotIndex = partition(nums, left, right);

    if (pivotIndex == targetIndex) {
        return nums[pivotIndex];
    } else if (pivotIndex < targetIndex) {
        return quickSelectHelper(nums, pivotIndex + 1, right, target
    } else {
```

```
            return quickSelectHelper(nums, left, pivotIndex - 1, targetI
        }
    }
```

2. Merge K Sorted Lists Optimization

```
// Optimized approach using divide and conquer
public ListNode mergeKLists(ListNode[] lists) {
    if (lists == null || lists.length == 0) return null;

    while (lists.length > 1) {
        List<ListNode> mergedLists = new ArrayList<>();

        for (int i = 0; i < lists.length; i += 2) {
            ListNode l1 = lists[i];
            ListNode l2 = (i + 1 < lists.length) ? lists[i + 1] : nu
            mergedLists.add(merge2Lists(l1, l2));
        }

        lists = mergedLists.toArray(new ListNode[0]);
    }

    return lists[0];
}

private ListNode merge2Lists(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0);
    ListNode current = dummy;

    while (l1 != null && l2 != null) {
        if (l1.val <= l2.val) {
            current.next = l1;
            l1 = l1.next;
        } else {
            current.next = l2;
            l2 = l2.next;
        }
        current = current.next;
    }

    current.next = (l1 != null) ? l1 : l2;
    return dummy.next;
}
```

> → **Key Idea**: Make locally optimal choice at each step to reach global optimum

> • **When to use**: Activity selection, coin change, jump games, interval scheduling

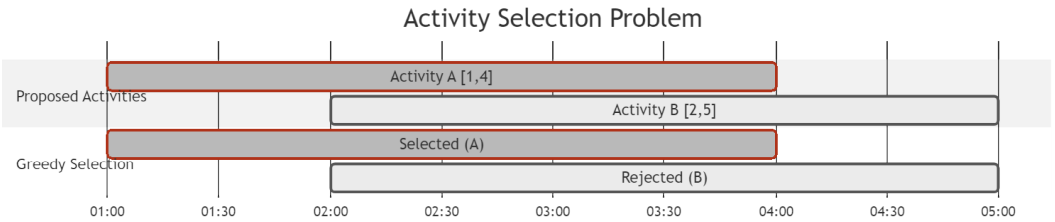
> ⊕ **Complexity**: Varies by problem, often O(n log n) due to sorting

> ⚠ **Common Pitfall**: Assuming greedy works without proving optimality or missing counterexamples

Overview

Visual Example

Activity Selection Timeline



> **Note**: Activity A is selected because it finishes earlier. Activity B is rejected because its start time (2) overlaps with A (ends at 4).

Core Implementation

Activity Selection Problem

```
import java.util.*;

public class ActivitySelection {
    static class Activity {
        int start, end;

        Activity(int start, int end) {
            this.start = start;
            this.end = end;
        }
```

```
    }

    /**
     * Select maximum number of non-overlapping activities
     */
    public static List<Activity> selectActivities(Activity[] activit
        // Sort by end time (greedy choice)
        Arrays.sort(activities, (a, b) -> a.end - b.end);

        List<Activity> selected = new ArrayList<>();
        selected.add(activities[0]); // Always select first activity

        int lastEndTime = activities[0].end;

        for (int i = 1; i < activities.length; i++) {
            // If activity starts after previous one ends, select it
            if (activities[i].start >= lastEndTime) {
                selected.add(activities[i]);
                lastEndTime = activities[i].end;
            }
        }

        return selected;
    }
}
```

Fractional Knapsack

```
import java.util.*;

public class FractionalKnapsack {
    static class Item {
        int weight, value;
        double ratio;

        Item(int weight, int value) {
            this.weight = weight;
            this.value = value;
            this.ratio = (double) value / weight;
        }
    }

    /**
     * Fractional knapsack using greedy approach
     */
    public static double fractionalKnapsack(Item[] items, int capaci
```

```
        // Sort by value-to-weight ratio (greedy choice)
        Arrays.sort(items, (a, b) -> Double.compare(b.ratio, a.ratio

        double totalValue = 0.0;
        int currentWeight = 0;

        for (Item item : items) {
            if (currentWeight + item.weight <= capacity) {
                // Take entire item
                currentWeight += item.weight;
                totalValue += item.value;
            } else {
                // Take fraction of item
                int remainingCapacity = capacity - currentWeight;
                totalValue += item.ratio * remainingCapacity;
                break;
            }
        }

        return totalValue;
    }
}
```

Huffman Coding

```
import java.util.*;

public class HuffmanCoding {
    static class Node implements Comparable<Node> {
        char character;
        int frequency;
        Node left, right;

        Node(char character, int frequency) {
            this.character = character;
            this.frequency = frequency;
        }

        Node(int frequency, Node left, Node right) {
            this.frequency = frequency;
            this.left = left;
            this.right = right;
        }

        @Override
        public int compareTo(Node other) {
```

```
            return this.frequency - other.frequency;
        }

        boolean isLeaf() {
            return left == null && right == null;
        }
    }

    /**
     * Build Huffman tree using greedy approach
     */
    public static Node buildHuffmanTree(Map<Character, Integer> freq
        PriorityQueue<Node> minHeap = new PriorityQueue<>();

        // Create leaf nodes
        for (Map.Entry<Character, Integer> entry : frequencies.entry
            minHeap.offer(new Node(entry.getKey(), entry.getValue())
        }

        // Build tree greedily
        while (minHeap.size() > 1) {
            Node left = minHeap.poll();
            Node right = minHeap.poll();

            Node merged = new Node(left.frequency + right.frequency,
            minHeap.offer(merged);
        }

        return minHeap.poll();
    }

    /**
     * Generate Huffman codes
     */
    public static Map<Character, String> generateCodes(Node root) {
        Map<Character, String> codes = new HashMap<>();
        generateCodesHelper(root, "", codes);
        return codes;
    }

    private static void generateCodesHelper(Node node, String code, M
        if (node.isLeaf()) {
            codes.put(node.character, code.isEmpty() ? "0" : code);
            return;
        }

        generateCodesHelper(node.left, code + "0", codes);
        generateCodesHelper(node.right, code + "1", codes);
```

```
    }
}
```

Key Optimizations

1. Sorting Optimization for Intervals

```
public class IntervalScheduling {

    /**
     * Optimized interval sorting with custom comparator
     */
    public static int maxNonOverlappingIntervals(int[][] intervals) {
        if (intervals.length == 0) return 0;

        // Sort by end time with tie-breaking by start time
        Arrays.sort(intervals, (a, b) -> {
            if (a[1] != b[1]) return a[1] - b[1]; // End time
            return a[0] - b[0]; // Start time for ties
        });

        int count = 1;
        int lastEnd = intervals[0][1];

        for (int i = 1; i < intervals.length; i++) {
            if (intervals[i][0] >= lastEnd) {
                count++;
                lastEnd = intervals[i][1];
            }
        }

        return count;
    }
}
```

2. Jump Game Optimizations

```
public class JumpGame {

    /**
     * Jump Game I - Can reach end?
```

```
     */
    public static boolean canJump(int[] nums) {
        int maxReach = 0;

        for (int i = 0; i < nums.length; i++) {
            if (i > maxReach) return false; // Can't reach this posi

            maxReach = Math.max(maxReach, i + nums[i]);

            if (maxReach >= nums.length - 1) return true;
        }

        return true;
    }

    /**
     * Jump Game II - Minimum jumps to reach end
     */
    public static int minJumps(int[] nums) {
        if (nums.length <= 1) return 0;

        int jumps = 0;
        int currentEnd = 0;
        int farthest = 0;

        for (int i = 0; i < nums.length - 1; i++) {
            farthest = Math.max(farthest, i + nums[i]);

            if (i == currentEnd) {
                jumps++;
                currentEnd = farthest;

                if (currentEnd >= nums.length - 1) break;
            }
        }

        return jumps;
    }
}
```

> → **Key Idea**: Break problem into subproblems and store results to avoid recomputation

> • **When to use**: Optimization problems, overlapping subproblems, optimal substructure

> ⊕ **Complexity**: Time/Space vary by problem dimension (1D: O(n), 2D: $O(n^2)$)

> ⚠ **Common Pitfall**: Not identifying optimal substructure or inefficient state representation

Overview

Dynamic Programming (DP) solves problems by breaking them into smaller subproblems and storing results to avoid redundant calculations. It works when problems have optimal substructure and overlapping subproblems.

Key steps: 1) Define state, 2) Identify base cases, 3) Find recurrence relation, 4) Determine computation order.

Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics
- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid
- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
// Template code for common variations
```

Time & Space Complexity

Problem Type	Time Complexity	Space Complexity

1D Linear DP	O(n)	O(n) → O(1) optimized
2D Grid DP	O(mn)	O(mn) → O(n) optimized
LCS/Edit Distance	O(mn)	O(mn) → O(min(m,n))
Knapsack	O(nW)	O(W)
LIS	$O(n^2)$ → O(n log n)	O(n)

Most DP problems can be space-optimized by observing that we often only need previous states.

← Previous: Greedy Algorithms | Back to Main Guide | Next: Graph Algorithms →

> → **Key Idea**: Model problems as graphs and use traversal/search algorithms to find solutions

> • **When to use**: Connectivity, shortest path, cycle detection, topological ordering

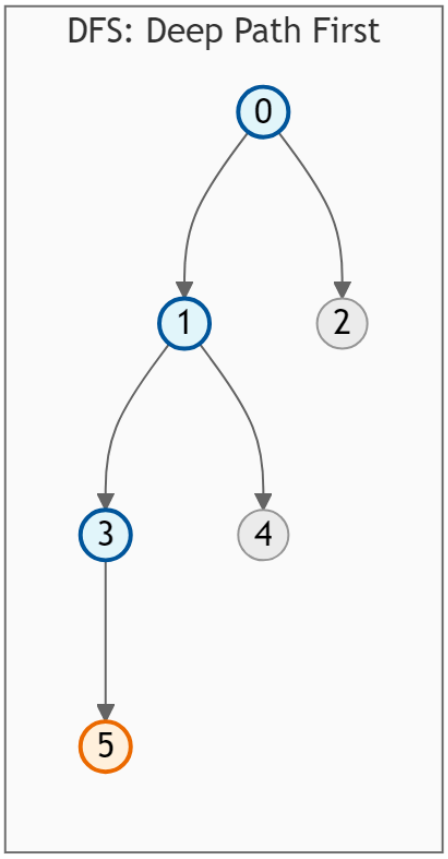
> ⊕ **Complexity**: BFS/DFS: O(V+E), Dijkstra: O(E log V), MST: O(E log V)

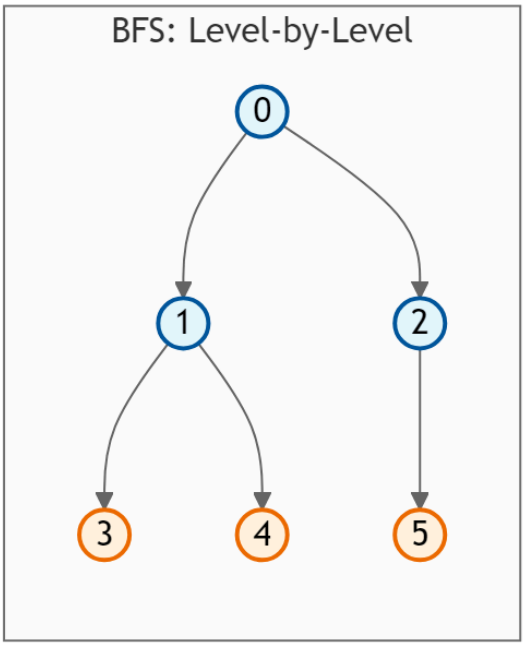
> ⚠ **Common Pitfall**: Not handling disconnected components or choosing wrong algorithm

Overview

Visual Example

BFS vs DFS Traversal Comparison





Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics
- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid
- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
// Template code for common variations
```

Time & Space Complexity

Algorithm	Time Complexity	Space Complexity
BFS/DFS	O(V + E)	O(V)
Dijkstra	O((V + E) log V)	O(V)
Bellman-Ford	O(VE)	O(V)
Kruskal MST	O(E log E)	O(V)
Prim MST	O(E log V)	O(V)

Where V = vertices, E = edges.

← Previous: Dynamic Programming | Back to Main Guide | Next: Tree Algorithms →

> → **Key Idea**: Leverage tree properties like hierarchy and recursion for efficient solutions

> • **When to use**: Tree traversal, BST operations, path problems, tree construction

> ⊕ **Complexity**: Most operations O(h) where h is height, O(n) for traversal

> ⚠ **Common Pitfall**: Not handling null nodes or unbalanced trees leading to O(n) height

Overview

Tree algorithms operate on hierarchical data structures. Key patterns include traversals, BST operations, path finding, and tree properties like height and diameter.

Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics
- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid
- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
// Template code for common variations
```

Time & Space Complexity

Operation	Time Complexity	Space Complexity
Tree Traversal	O(n)	O(h)
BST Search	O(log n) avg, O(n) worst	O(h)
Tree Construction	O(n)	O(n)
Path Problems	O(n)	O(h)

Where n = number of nodes, h = height of tree.

← Previous: Graph Algorithms | Back to Main Guide | Next: Backtracking →

> → **Key Idea**: Explore solution space recursively and backtrack when constraints are violated

> • **When to use**: Permutations, combinations, constraint satisfaction, puzzle solving

> ⊕ **Complexity**: Exponential in worst case, depends on pruning effectiveness

> ⚠ **Common Pitfall**: Not properly backtracking state or missing pruning opportunities

Overview

Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics
- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid
- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
// Template code for common variations
```

← Previous: Tree Algorithms | Back to Main Guide | Next: Trie & String Algorithms →

> → **Key Idea**: Use prefix trees and pattern matching for efficient string operations

> • **When to use**: Prefix matching, autocomplete, pattern searching, dictionary operations

> ⊕ **Complexity**: Trie operations O(m) where m is string length, KMP O(n+m)

> ⚠ **Common Pitfall**: Not handling memory efficiently in trie or implementing pattern matching incorrectly

Overview

Visual Example

Trie Structure Visualization

```
Words: ["cat", "car", "card", "dog", "dodge"]

Trie Structure:
                root
               /    \
              c      d
              |      |
              a      o
              |      |
              t      g
             /|\    /|\
            ★ r e  ★ d
              |      |
              ★      g
              |      |
              d      e
              |      |
              ★      ★

★ = End of word marker

>> Prefix "ca" shared by "cat", "car", "card"
>> Prefix "dog" shared by "dog", "dodge"
```

Trie Search Process

```
Search for "car":

Step 1: Start at root
Step 2: Follow 'c' → found
Step 3: Follow 'a' → found
```

```
Step 4: Follow 'r' → found + end marker ★
Result: "car" exists in trie ✓

Search for "care":
Step 1-3: Same as above
Step 4: Follow 'e' → not found ✗
Result: "care" does not exist

>> Search complexity: O(length of word)
```

KMP Failure Function

```
Pattern: "ABABCABAB"
Text:    "ABABDABABCABABCABAB"

Build failure function for pattern:
A B A B C A B A B
0 0 1 2 0 1 2 3 4
│ │ │ │ │ │ │ │ │
└─┼─┘ │ │ └─┼─┘ │
  └───┘ │   └───┘
        │
   No prefix match

KMP Search with failure function:
Text:    A B A B D A B A B C A B A B C A B A B
Pattern: A B A B C A B A B
         ↑ ↑ ↑ ↑ ✗
         Use failure function to skip ahead instead of restarting

⚡ Avoids redundant character comparisons
```

Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics
- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid

- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
// Template code for common variations
```

← Previous: Backtracking | Back to Main Guide | Next: Math & Bit Manipulation →

> → **Key Idea**: Use mathematical properties and bit operations for efficient computation

> • **When to use**: Number theory, prime numbers, bit operations, mathematical optimization

> ⨁ **Complexity**: Varies by algorithm, bit operations typically O(1) or O(log n)

> ⚠ **Common Pitfall**: Integer overflow, incorrect bit manipulation, or not using mathematical shortcuts

Overview

Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics
- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid

- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
// Template code for common variations
```

← Previous: Trie & String Algorithms | Back to Main Guide | Next: Java Data Structures →

> → **Key Idea**: Master Java's built-in data structures for efficient interview problem solving

> • **When to use**: Interview implementation, understanding performance characteristics, choosing right collections

> ⨁ **Complexity**: Varies by data structure, HashMap O(1), TreeMap O(log n), ArrayList O(1) amortized

> ⚠ **Common Pitfall**: Using wrong collection type or not understanding performance implications

Overview

Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics
- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid

- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
// Template code for common variations
```

← Previous: Math & Bit Manipulation | Back to Main Guide | Next: Flow Algorithms →

> → **Key Idea**: Model problems as networks with capacity constraints to find optimal flow

> • **When to use**: Maximum flow, minimum cut, bipartite matching, assignment problems

> ⊕ **Complexity**: $O(V^2E)$ for Ford-Fulkerson, $O(V^3)$ for maximum bipartite matching

> ⚠ **Common Pitfall**: Not handling residual graphs correctly or missing augmenting paths

Overview

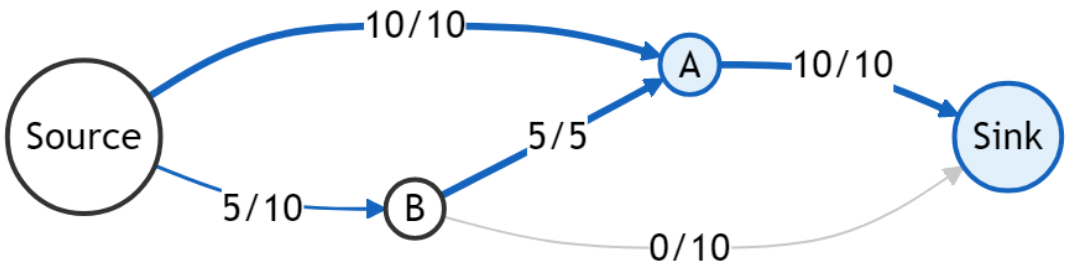
Flow algorithms solve optimization problems on networks by finding maximum flow from source to sink while respecting capacity constraints. Key applications include network routing, resource allocation, and matching problems.

Flow networks consist of:

- **Vertices**: Nodes in the network
- **Edges**: Connections with capacity constraints
- **Source**: Starting point of flow
- **Sink**: Destination point of flow
- **Capacity**: Maximum flow allowed through an edge

Visual Example

Maximum Flow Network



> Values represent Flow/Capacity. Thicker blue lines show the main flow path augmenting from Source to Sink.

Core Implementation

Ford-Fulkerson Algorithm (DFS-based)

```
import java.util.*;

public class MaxFlow {
    static class Edge {
        int to, capacity, flow;

        Edge(int to, int capacity) {
            this.to = to;
            this.capacity = capacity;
            this.flow = 0;
        }

        int residualCapacity() {
            return capacity - flow;
        }
    }

    private List<List<Edge>> graph;
    private boolean[] visited;

    public MaxFlow(int n) {
        graph = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
```

```
        }
        visited = new boolean[n];
    }

    public void addEdge(int from, int to, int capacity) {
        graph.get(from).add(new Edge(to, capacity));
        graph.get(to).add(new Edge(from, 0)); // Reverse edge for re
    }

    /**
     * Ford-Fulkerson algorithm using DFS
     */
    public int maxFlow(int source, int sink) {
        int totalFlow = 0;

        while (true) {
            Arrays.fill(visited, false);
            int pathFlow = dfs(source, sink, Integer.MAX_VALUE);

            if (pathFlow == 0) break; // No augmenting path found

            totalFlow += pathFlow;
        }

        return totalFlow;
    }

    private int dfs(int node, int sink, int minCapacity) {
        if (node == sink) return minCapacity;

        visited[node] = true;

        for (Edge edge : graph.get(node)) {
            if (!visited[edge.to] && edge.residualCapacity() > 0) {
                int pathFlow = dfs(edge.to, sink,
                    Math.min(minCapacity, edge.residualCapacity()));

                if (pathFlow > 0) {
                    edge.flow += pathFlow;

                    // Update reverse edge
                    for (Edge reverseEdge : graph.get(edge.to)) {
                        if (reverseEdge.to == node) {
                            reverseEdge.flow -= pathFlow;
                            break;
                        }
                    }

                    return pathFlow;
```

```
                }
            }
        }

        return 0;
    }
}
```

Edmonds-Karp Algorithm (BFS-based)

```
public class EdmondsKarp {
    private int[][] capacity;
    private int[][] flow;
    private int[] parent;
    private int n;

    public EdmondsKarp(int n) {
        this.n = n;
        capacity = new int[n][n];
        flow = new int[n][n];
        parent = new int[n];
    }

    public void addEdge(int from, int to, int cap) {
        capacity[from][to] = cap;
    }

    /**
     * Edmonds-Karp algorithm using BFS
     */
    public int maxFlow(int source, int sink) {
        int totalFlow = 0;

        while (bfs(source, sink)) {
            int pathFlow = Integer.MAX_VALUE;

            // Find minimum capacity along the path
            for (int v = sink; v != source; v = parent[v]) {
                int u = parent[v];
                pathFlow = Math.min(pathFlow, capacity[u][v] - flow[
            }

            // Update flow along the path
            for (int v = sink; v != source; v = parent[v]) {
                int u = parent[v];
                flow[u][v] += pathFlow;
```

```
                flow[v][u] -= pathFlow; // Reverse edge
            }

            totalFlow += pathFlow;
        }

        return totalFlow;
    }

    private boolean bfs(int source, int sink) {
        boolean[] visited = new boolean[n];
        Queue<Integer> queue = new LinkedList<>();

        queue.offer(source);
        visited[source] = true;
        parent[source] = -1;

        while (!queue.isEmpty()) {
            int u = queue.poll();

            for (int v = 0; v < n; v++) {
                if (!visited[v] && capacity[u][v] > flow[u][v]) {
                    queue.offer(v);
                    visited[v] = true;
                    parent[v] = u;
                    if (v == sink) return true;
                }
            }
        }

        return false;
    }
}
```

Key Optimizations

1. Residual Graph Construction

```
// Efficient residual graph using edge list
static class EdgeList {
    List<Edge> edges;
    List<List<Integer>> graph;

    void addEdge(int from, int to, int capacity) {
```

```
        graph.get(from).add(edges.size());
        edges.add(new Edge(to, capacity));

        graph.get(to).add(edges.size());
        edges.add(new Edge(from, 0)); // Reverse edge
    }

    int residualCapacity(int edgeId) {
        return edges.get(edgeId).capacity - edges.get(edgeId).flow;
    }
}
```

2. Level-based BFS (Dinic's Algorithm)

```
public class Dinic {
    private int[] level;
    private int[] iter;

    private boolean bfs(int source, int sink) {
        Arrays.fill(level, -1);
        level[source] = 0;

        Queue<Integer> queue = new LinkedList<>();
        queue.offer(source);

        while (!queue.isEmpty()) {
            int u = queue.poll();

            for (int edgeId : graph.get(u)) {
                Edge edge = edges.get(edgeId);

                if (level[edge.to] == -1 && edge.residualCapacity() 
                    level[edge.to] = level[u] + 1;
                    queue.offer(edge.to);
                }
            }
        }

        return level[sink] != -1;
    }
}
```